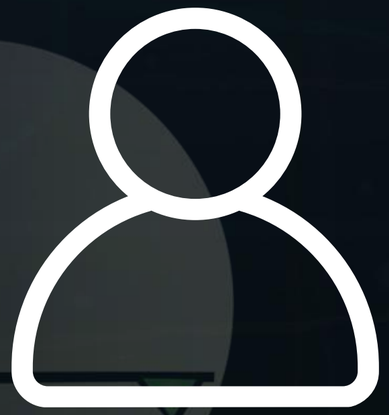
DATASENSE
LET THE DATA TALK

HOW TO SCALE AN APPLICATION

SCALE FROM ZERO TO MILLION USERS



Today

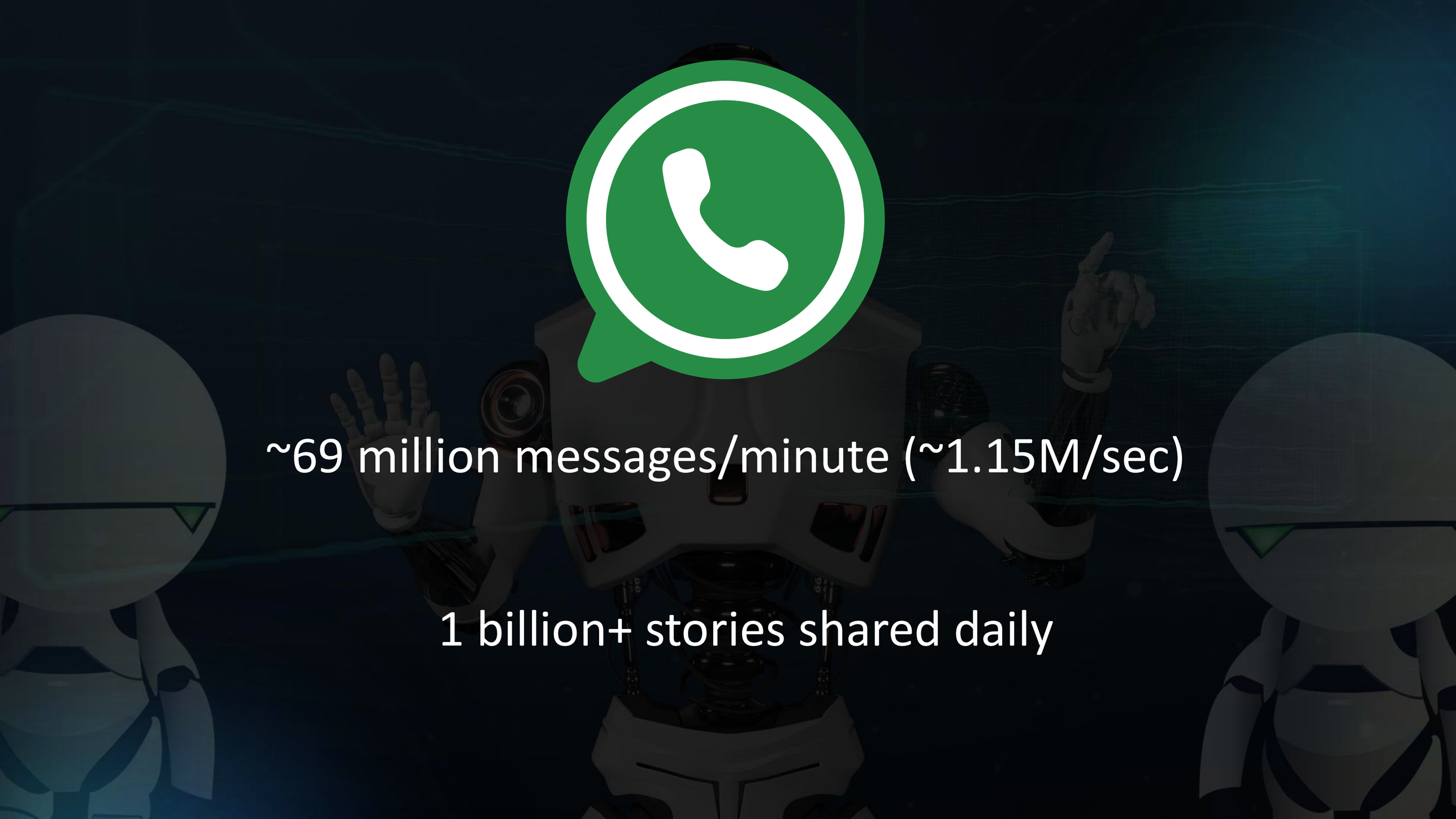
Tomorrow

99,000 searches/sec

Handles ~8.5 billion searches per day

4,000 posts liked/sec

1 billion+ stories shared daily



~69 million messages/minute (~1.15M/sec)

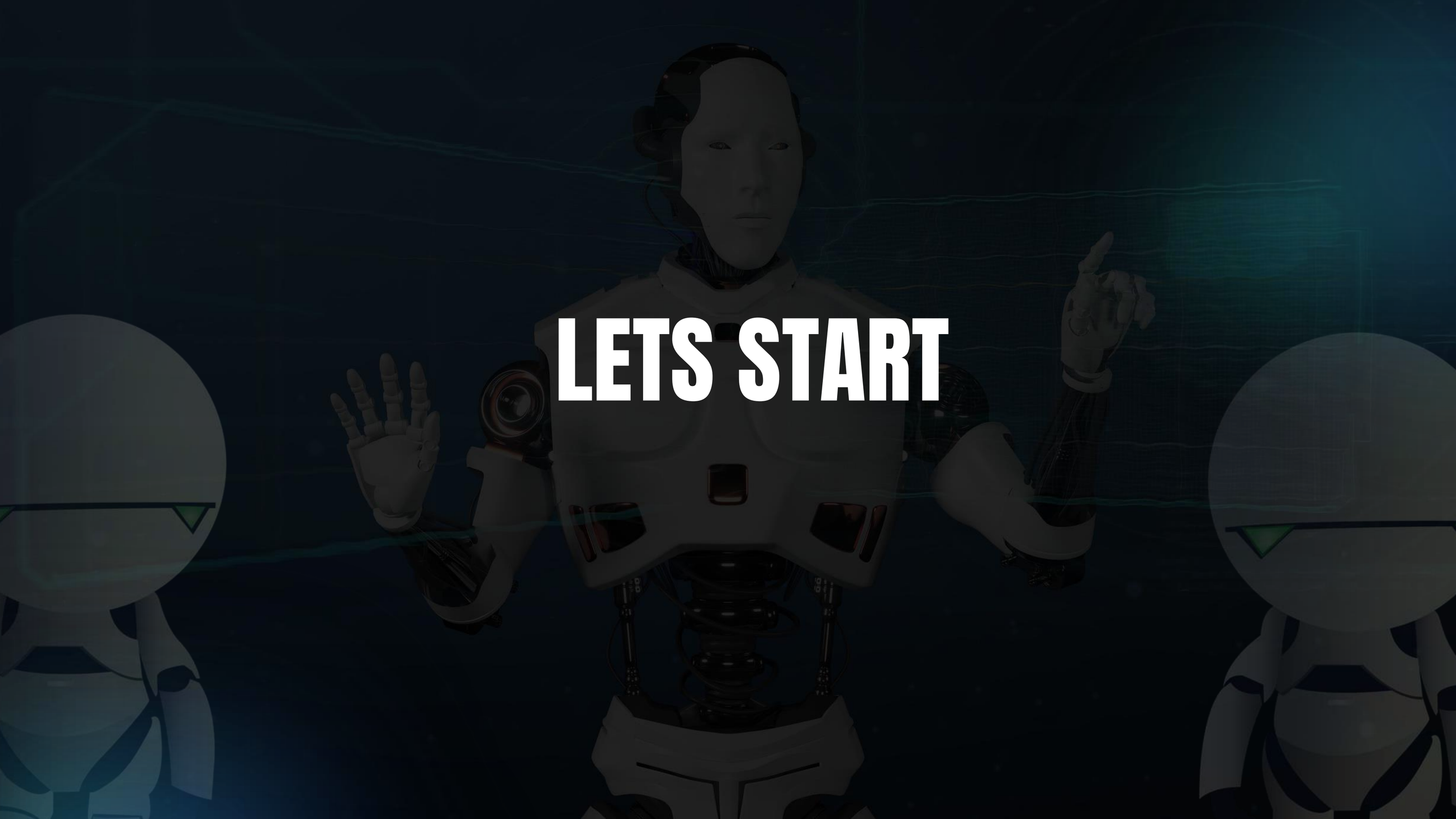
1 billion+ stories shared daily

WHAT WE SEE IS TIP OF ICEBERG

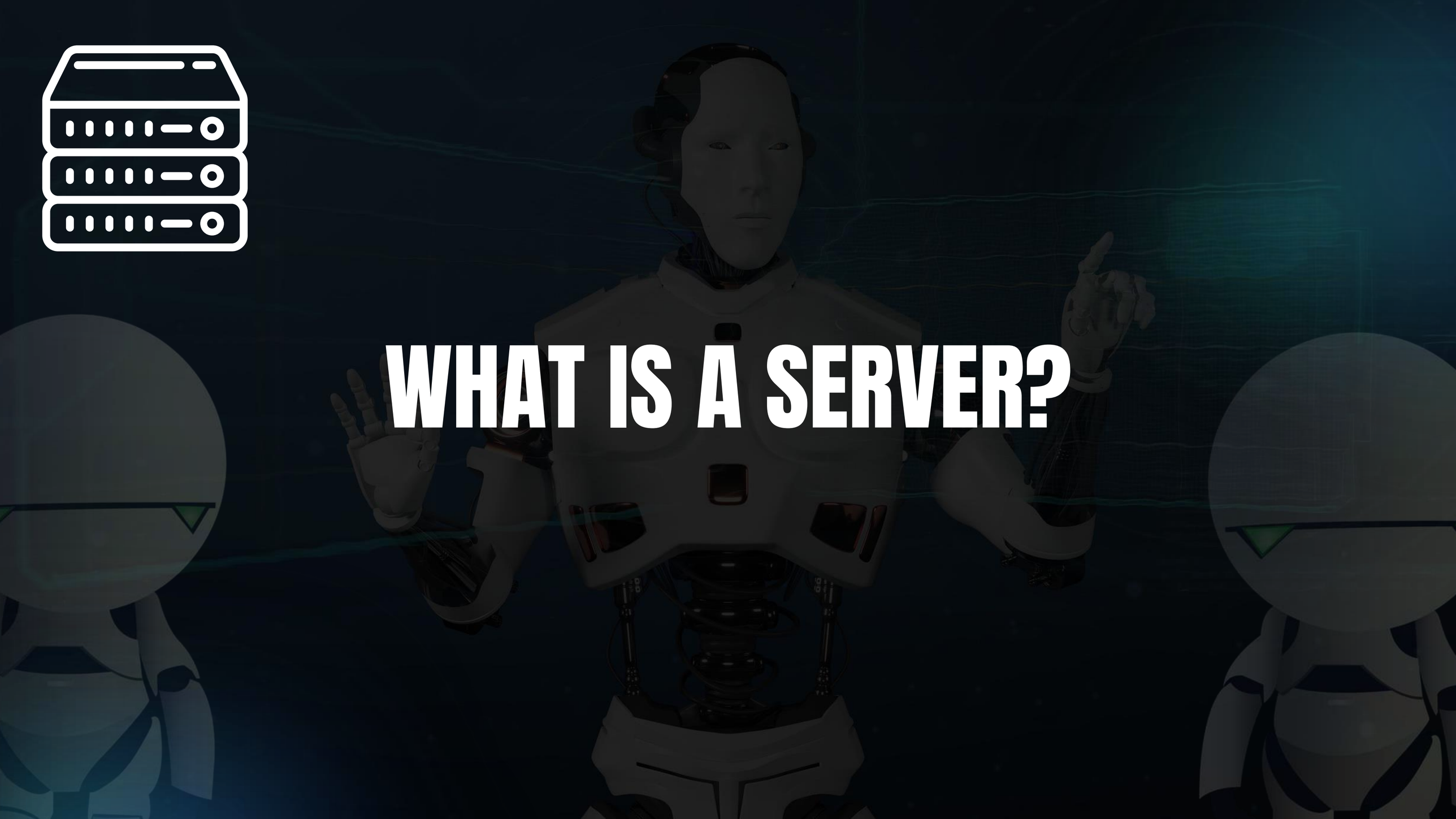
- “YOU VISIT A WEBSITE”
- “YOU WATCH A VIDEO”
- “YOU SEND A MESSAGE”
- (CLEAN UI, INSTANT RESPONSE)

REALITY

- 🌐 LOAD BALANCERS
- 🧱 MICROSERVICES
- ☁️ CLOUD AUTO-SCALING GROUPS
- 🧠 CACHING SYSTEMS (REDIS, CDN)
- 💾 DISTRIBUTED DATABASES
- 🧭 SERVICE DISCOVERY
- 📈 MONITORING & FAILOVER SYSTEMS
- 🧯 DISASTER RECOVERY & BACKUPS
- 🔒 SECURITY, API GATEWAYS, RATE LIMITERS


LETS START

1.SINGLE SERVER SETUP



WHAT IS A SERVER?

A SERVER IS A COMPUTER OR SYSTEM THAT PROVIDES RESOURCES, DATA, SERVICES, OR PROGRAMS TO OTHER COMPUTERS — CALLED CLIENTS — OVER A NETWORK.

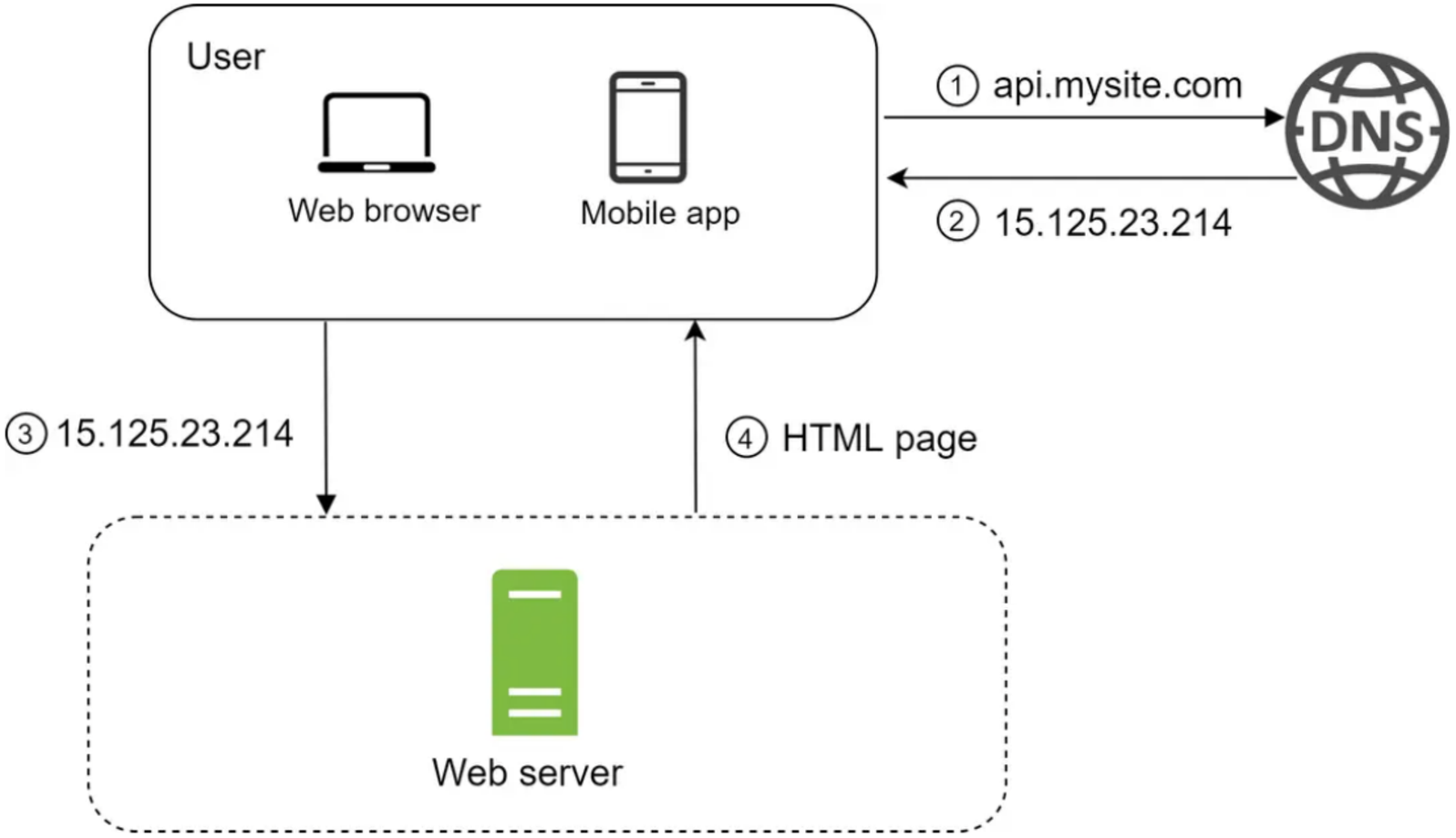
IF YOU'VE EVER OPENED A WEBSITE, WATCHED A YOUTUBE VIDEO, OR SENT A MESSAGE, YOU'VE USED A SERVER.

- YOUR DEVICE: "HEY, I WANT THIS CONTENT."
- SERVER: "HERE IT IS."

A SERVER IS JUST A SPECIALIZED COMPUTER (OFTEN STRONGER THAN A PERSONAL LAPTOP) THAT:

- WAITS FOR REQUESTS (LIKE PAGE LOADS OR API CALLS)
- PROCESSES THOSE REQUESTS (FETCHES INFO, RUNS LOGIC)
- RESPONDS WITH RESULTS (LIKE HTML PAGES, DATA, FILES)

Server Type	Purpose
Web Server	Delivers websites (e.g., Apache, Nginx)
Database Server	Stores and serves structured data (e.g., MySQL, MongoDB)
File Server	Shares files and folders over a network
Application Server	Runs business logic for apps
Mail Server	Handles email sending and receiving

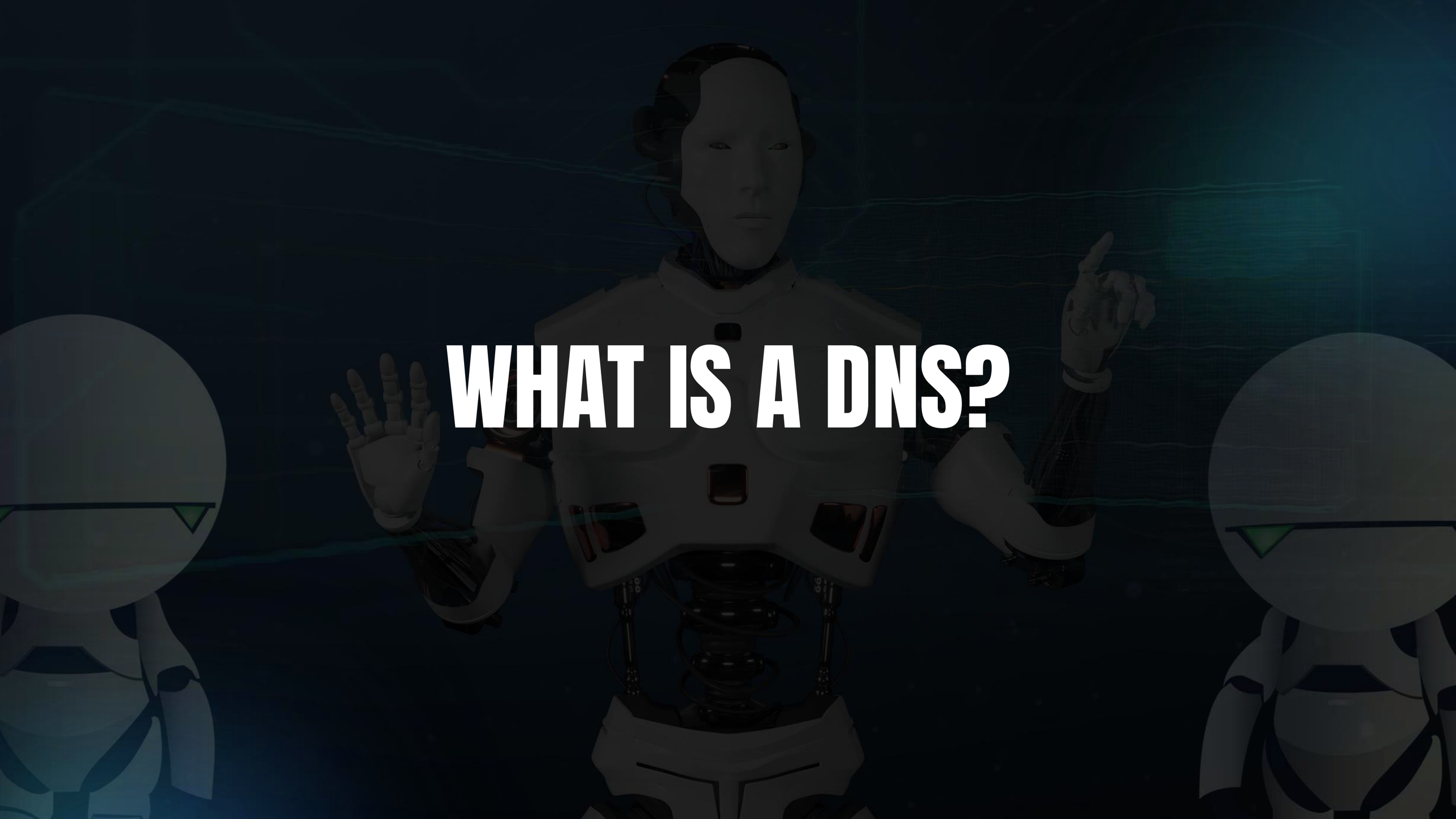

User
Web browser
Mobile app
① api.mysite.com
DNS
② 15.125.23.214
③ 15.125.23.214
④ HTML page
Web server

1. Users access websites through domain names, such as api.mys
ite.com. Usually, the Domain Name System (DNS) is a paid service provided by 3rd parties and not hosted by our servers.

2. Internet Protocol (IP) address is returned to the browser or mobile app. In the example, IP address 15.125.23.214 is returned.

3. Once the IP address is obtained, Hypertext Transfer Protocol (HTTP) [1] requests are sent directly to your web server.

4. The web server returns HTML pages or JSON response for rendering.



WHAT IS A DNS?

DNS? (Domain Name System)

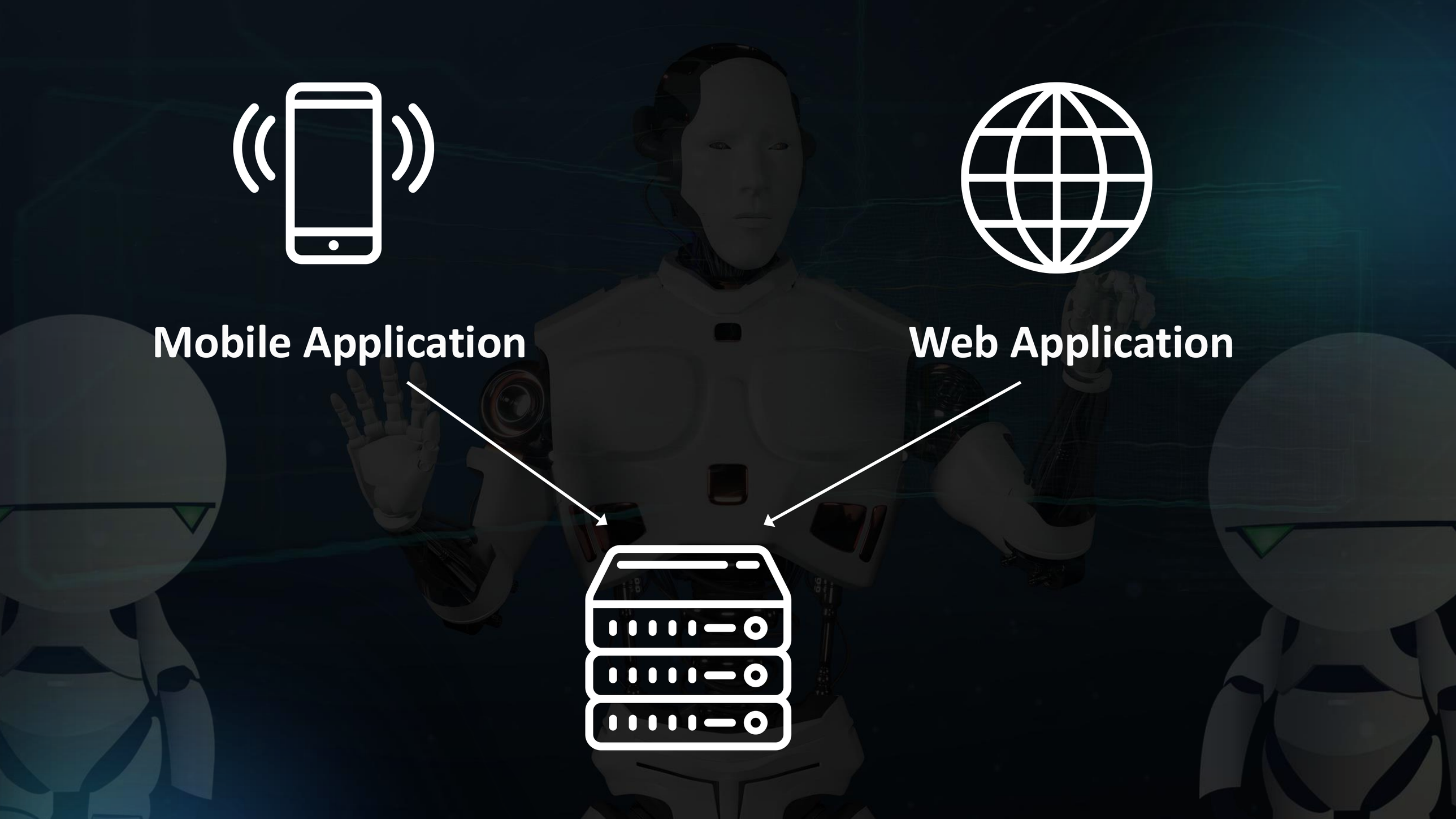
DNS is like the phonebook of the internet.

When you type a website like www.google.com into your browser, DNS translates that name into an IP address like 142.250.190.132, which is the actual address of the server where the site lives.

How DNS Stores and Resolves Values

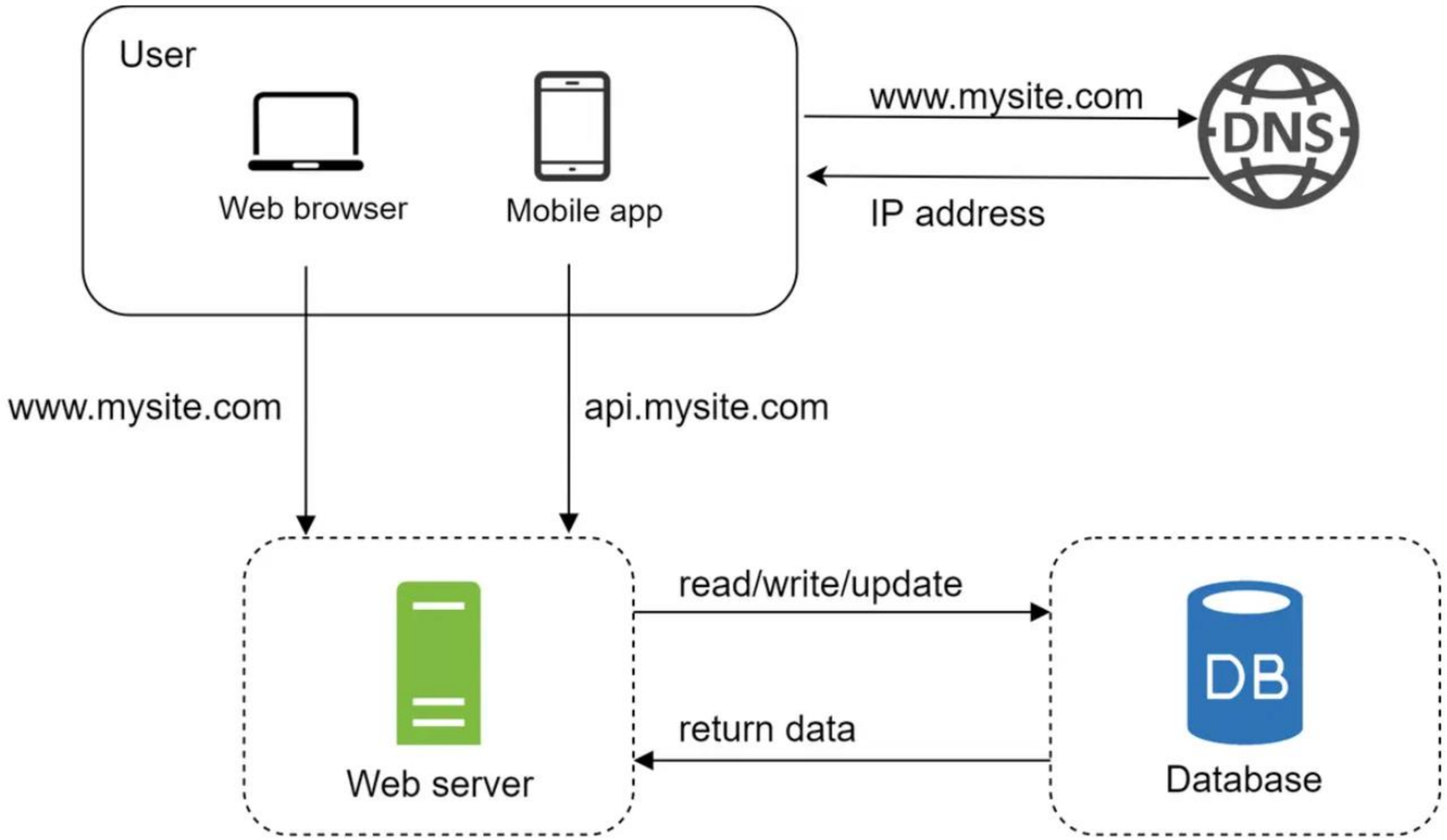
Step-by-Step DNS Resolution:

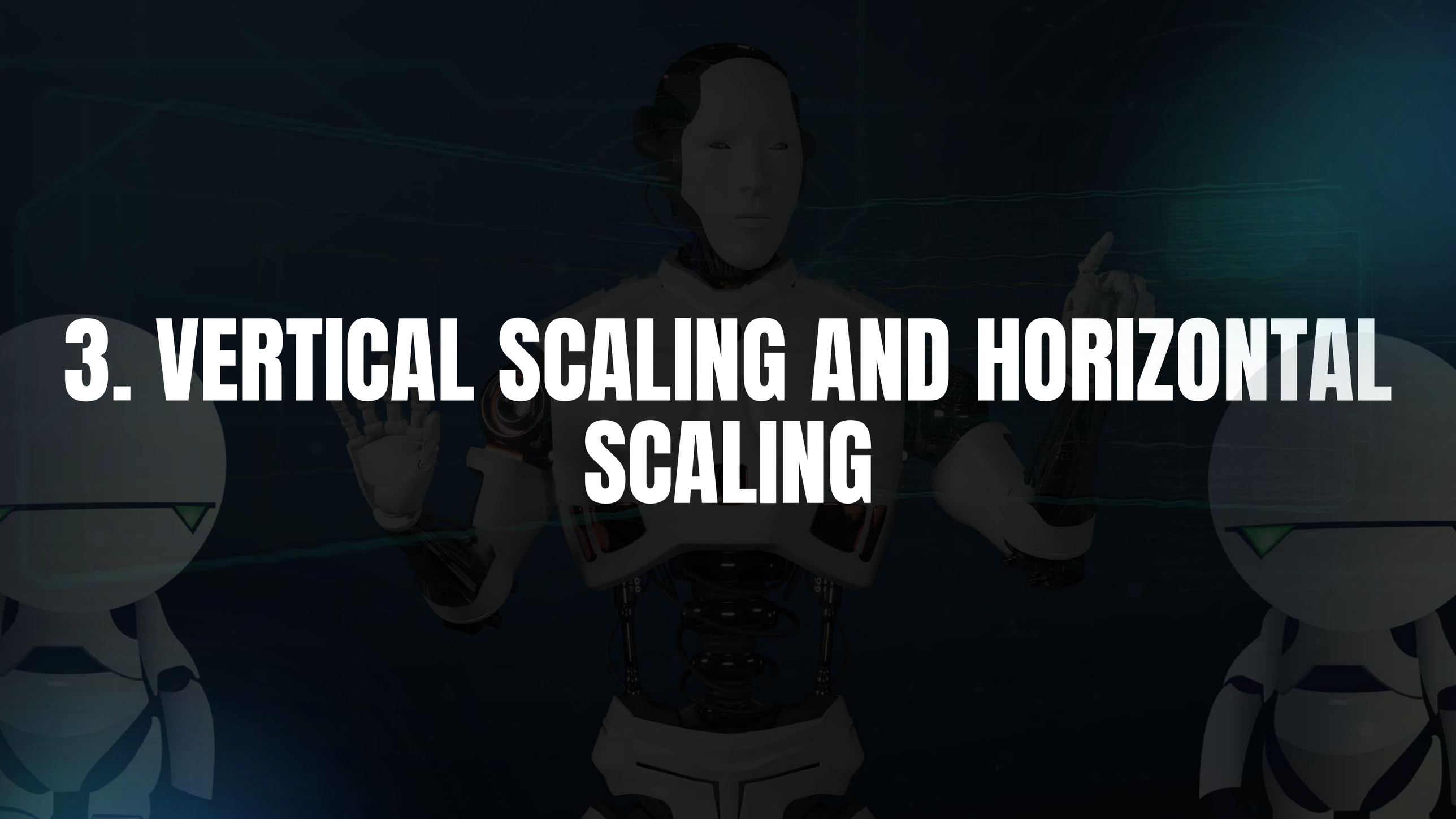
1. You type example.com in your browser.
2. Browser checks if it’s already cached (stored locally).
3. If not, it asks a DNS resolver (usually provided by your ISP).
4. The resolver asks the Root DNS Server – “Where can I find .com domains?”
5. It then asks a TLD Server (Top-Level Domain server) – “Where is example.com?”
6. Finally, it asks the Authoritative Name Server – “What is the IP of example.com?”
7. The IP address is returned and used to connect to the web server.
8. Your browser loads the website from that IP.


Mobile Application
Web Application

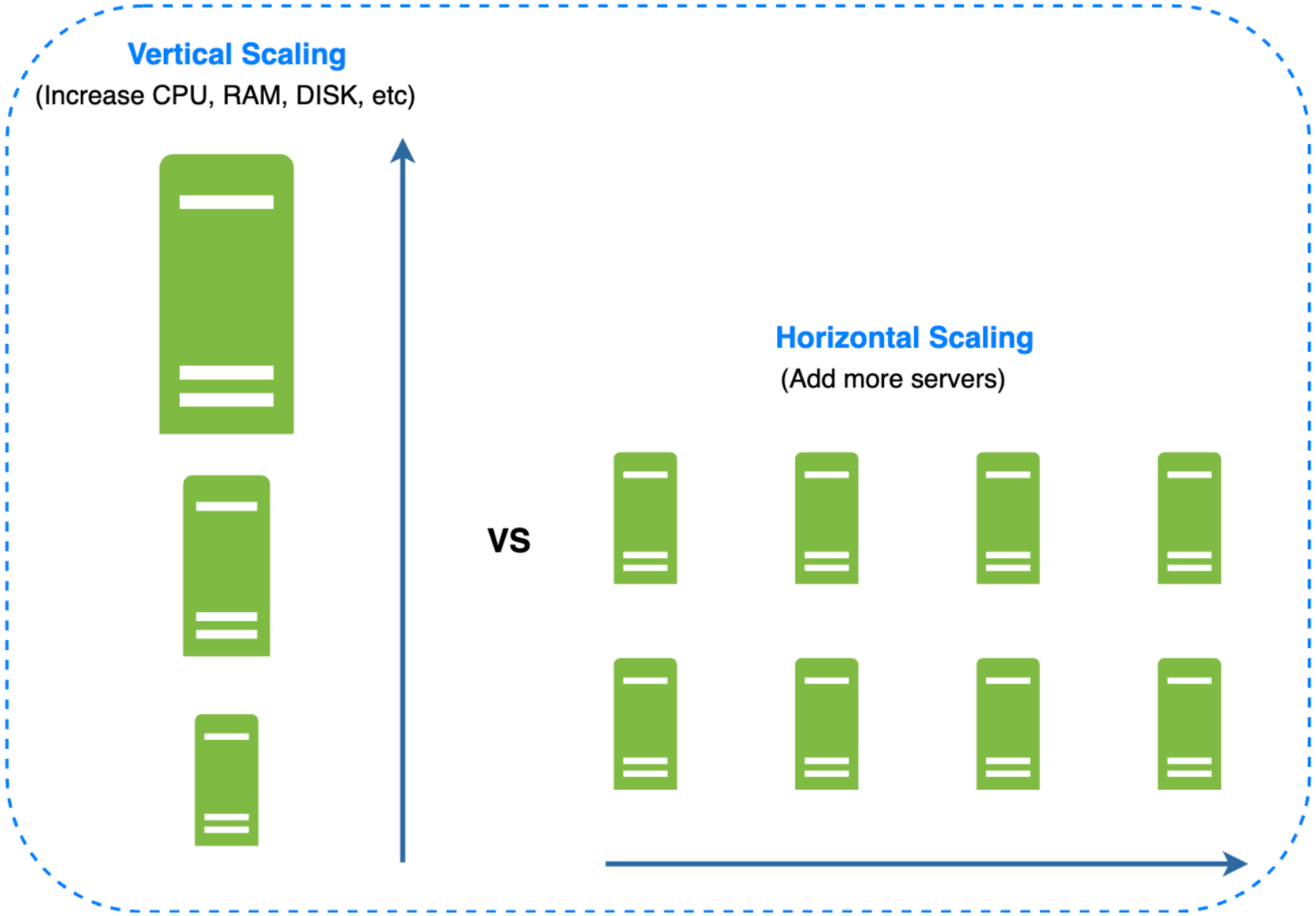
```
{
    "id":12,
    "firstName":"John",
    "lastName":"Smith",
    "address":{
        "streetAddress":"21 2nd Street",
        "city":"New York",
        "state":"NY",
        "postalCode":10021
    },
    "phoneNumbers":[
        "212 555-1234",
        "646 555-4567"
    ]
}
```

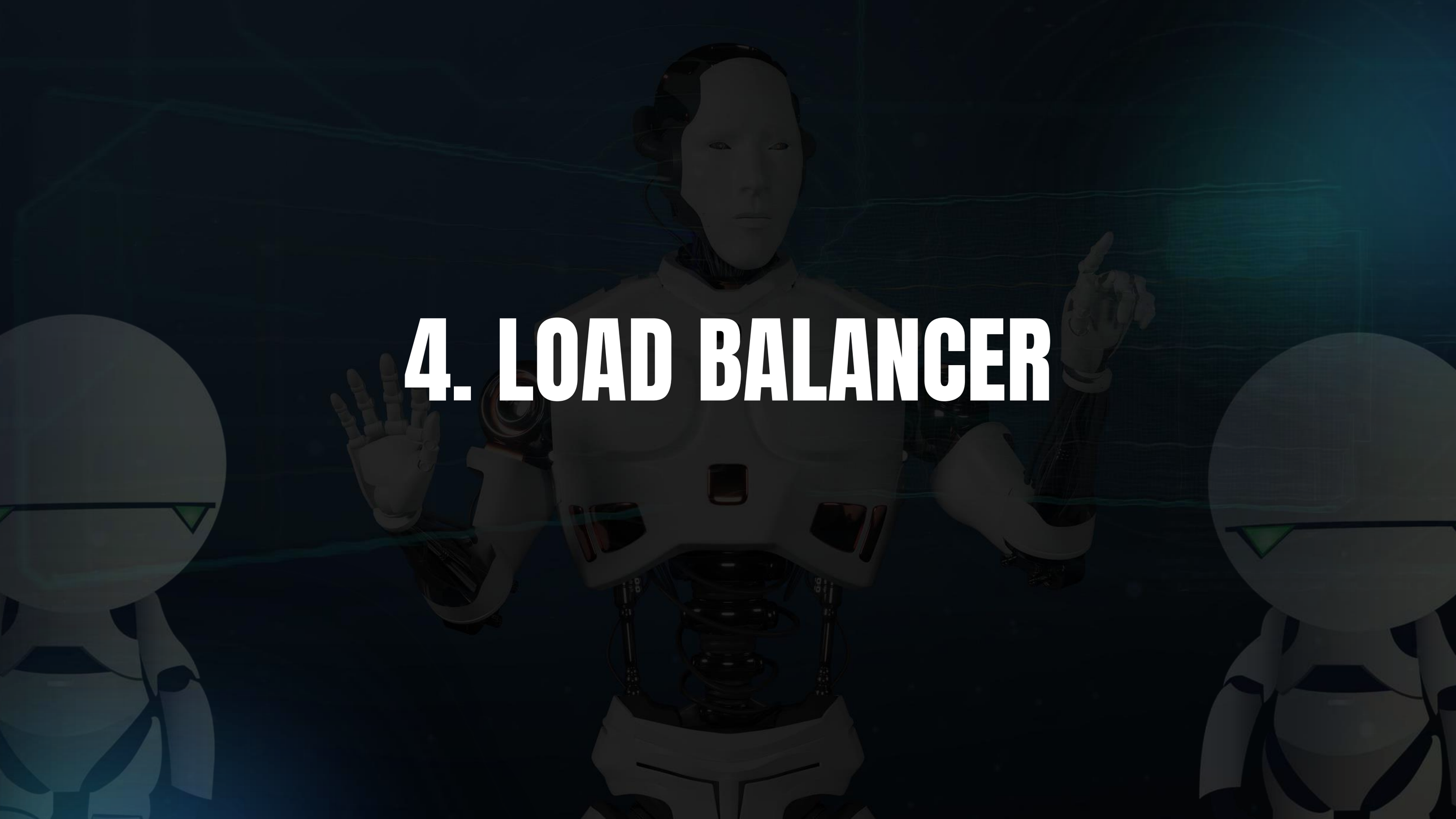
2. DATABASE


User
Web browser
Mobile app
www.mysite.com
DNS
IP address
www.mysite.com
api.mysite.com
read/write/update
return data
Web server
DB
Database

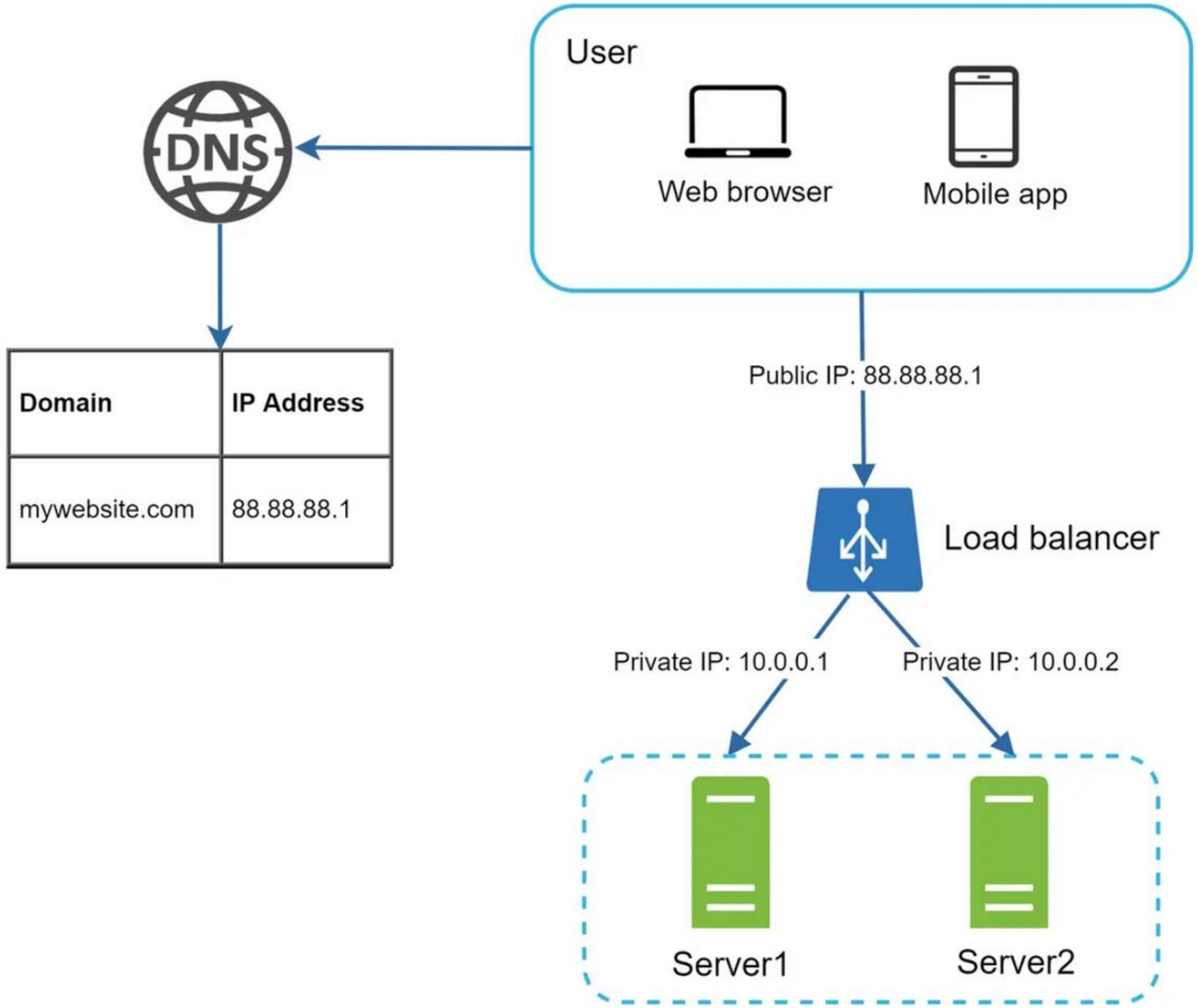


3. VERTICAL SCALING AND HORIZONTAL SCALING


Vertical Scaling
(Increase CPU, RAM, DISK, etc)
VS
Horizontal Scaling
(Add more servers)



4. LOAD BALANCER

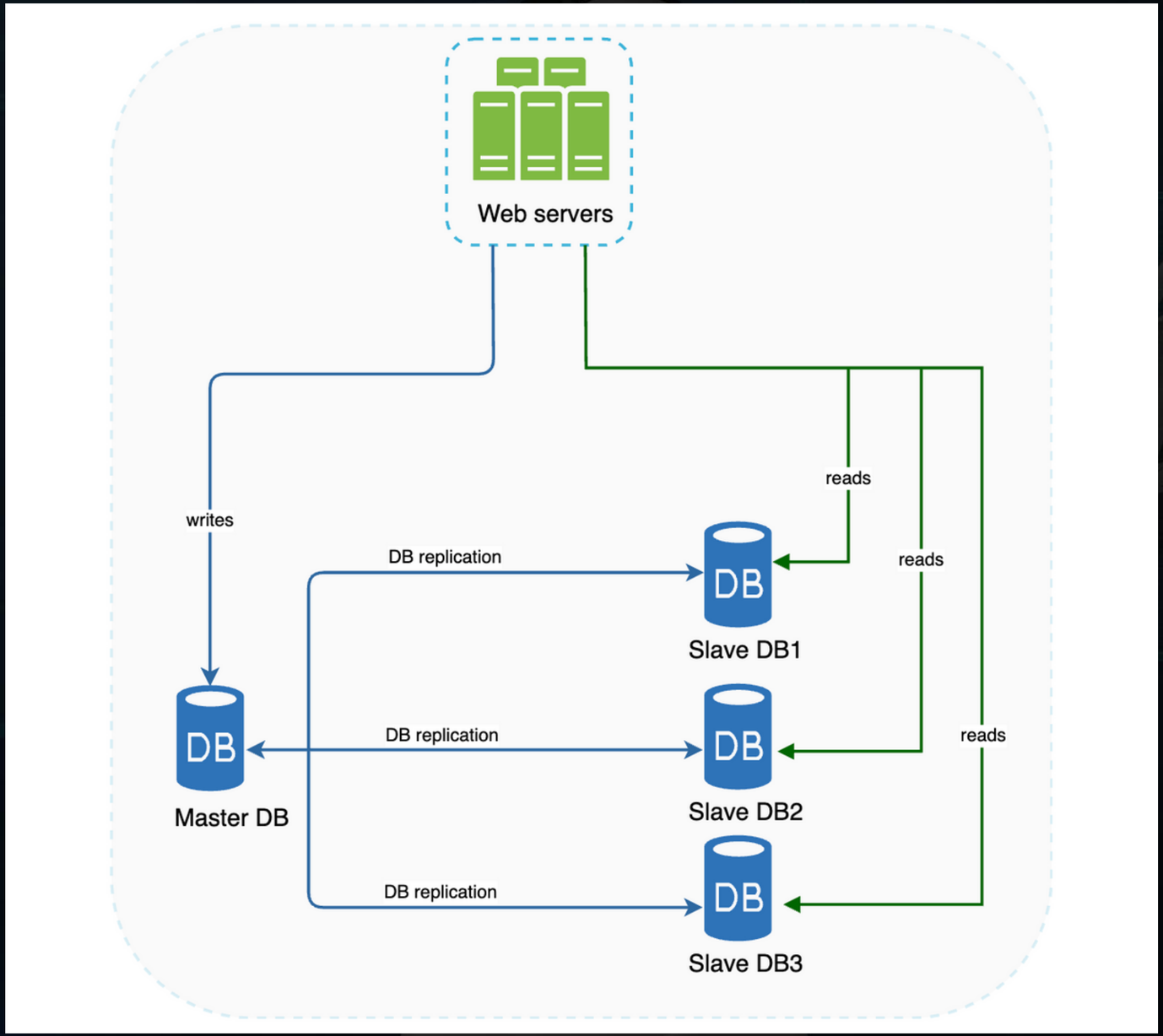


Domain	IP Address
mywebsite.com	88.88.88.1

LOAD BALANCER

- Users connect to the load balancer’s public IP, not directly to web servers — this hides and protects internal servers.
- Web servers use private IPs, making them unreachable from the internet and improving security.
- Load balancer routes traffic via private IPs to available web servers behind the scenes.
- If one server fails, traffic is automatically routed to a healthy one — ensuring failover and high availability.
- Easily scale traffic by adding more web servers; the load balancer distributes traffic to all healthy servers.

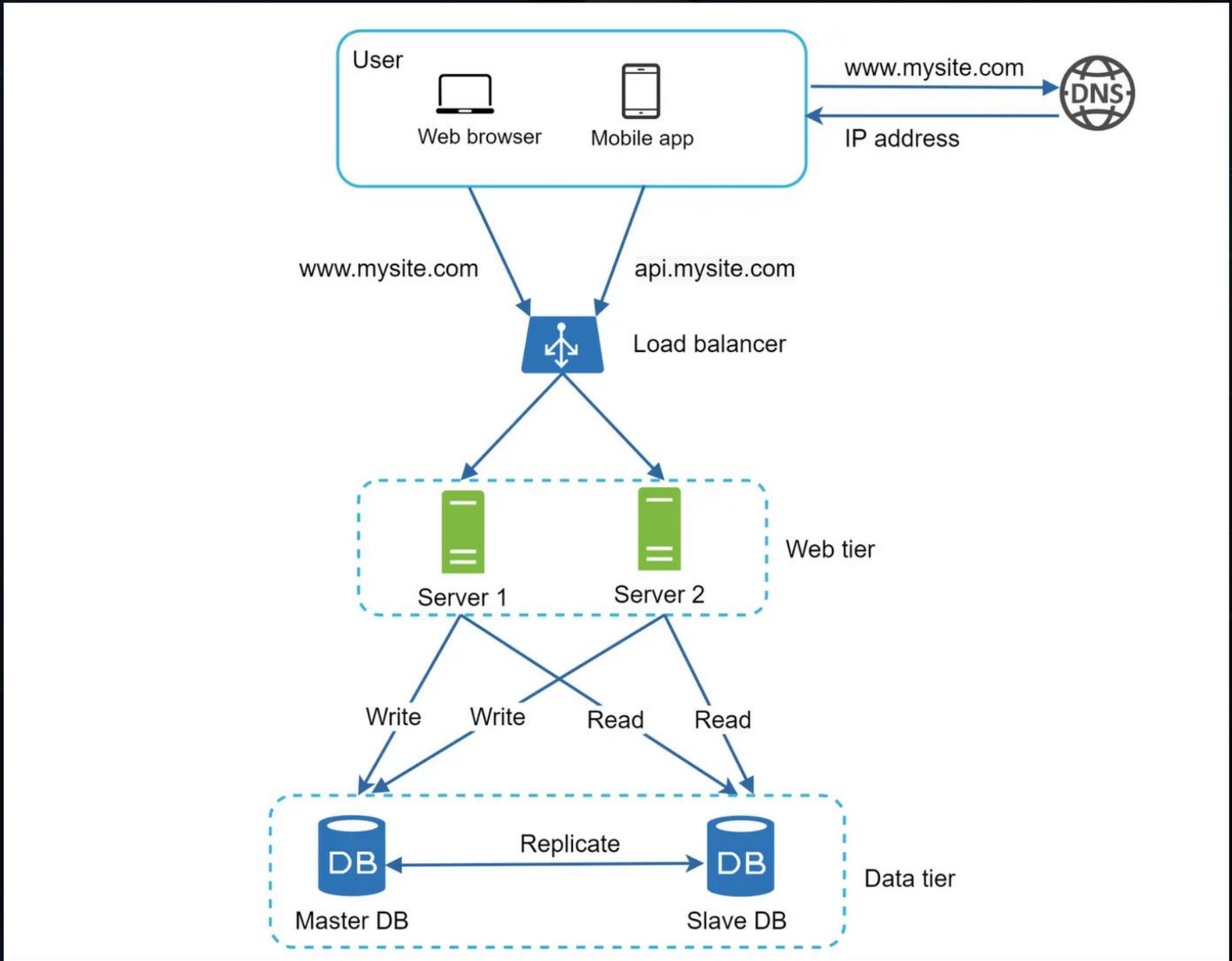
5. DATBASE REPLICATION


Web servers
writes
reads
reads
reads
DB replication
DB replication
DB replication
DB
Master DB
DB
Slave DB1
DB
Slave DB2
DB
Slave DB3

Database Replication

- **Master-slave architecture:** The master handles all write operations (insert, update, delete), while slaves handle read-only operations to distribute load.
- **Performance boost:** Reads are spread across slave databases, enabling parallel query processing and faster response times.
- **Reliability:** Data is replicated across multiple servers, ensuring no data loss even during disasters like earthquakes or outages.
- **High availability:** If one slave goes offline, reads are rerouted to healthy replicas or temporarily to the master until a replacement is added.
- **Failover mechanism:** If the master goes offline, a slave can be promoted to master, maintaining uninterrupted operation.

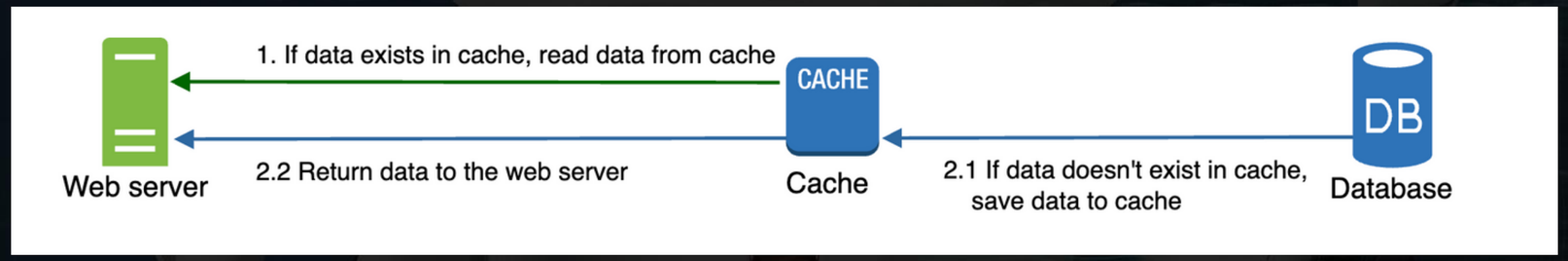
DESIGN SO FAR..


User
Web browser
Mobile app
www.mysite.com
DNS
IP address
www.mysite.com
api.mysite.com
Load balancer
Server 1
Server 2
Web tier
Write
Write
Read
Read
DB
Replicate
DB
Master DB
Slave DB
Data tier

6. CACHE

Cache

A cache is a temporary storage area that stores the result of expensive responses or frequently accessed data in memory so that subsequent requests are served more quickly. Every time a new web page loads, one or more database calls are executed to fetch data. The application performance is greatly affected by calling the database repeatedly

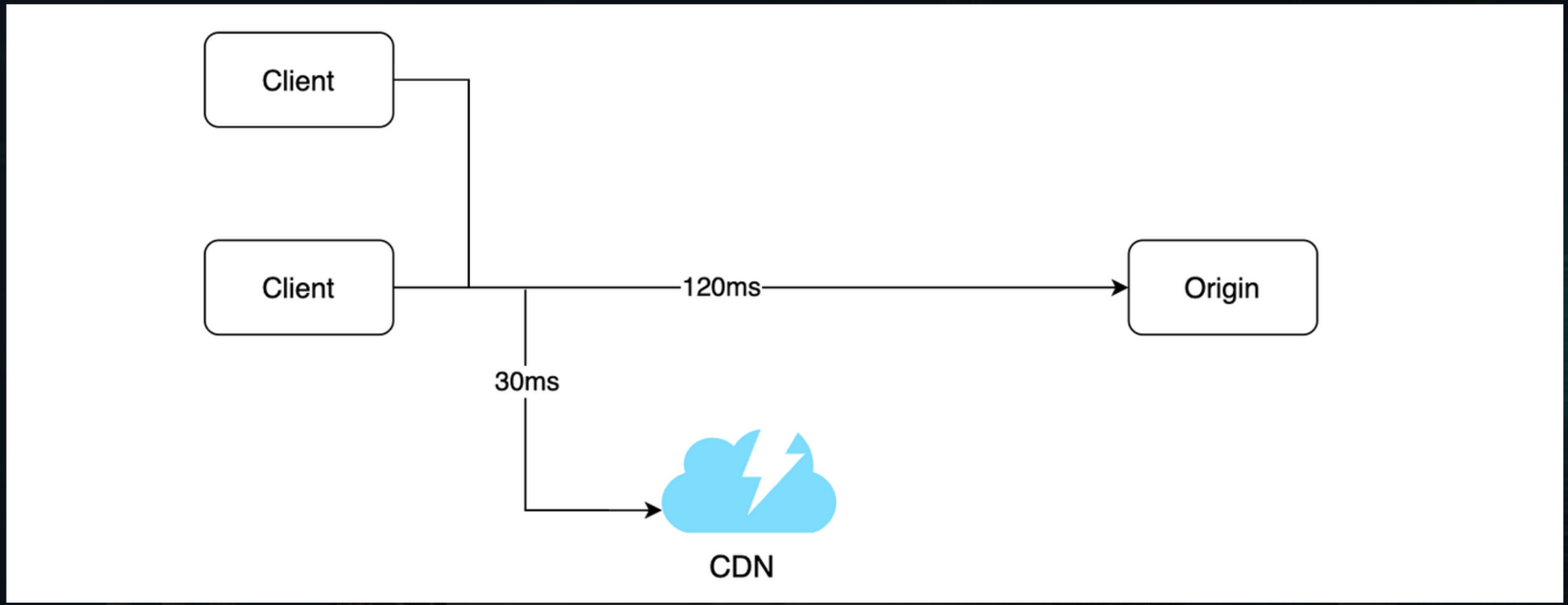


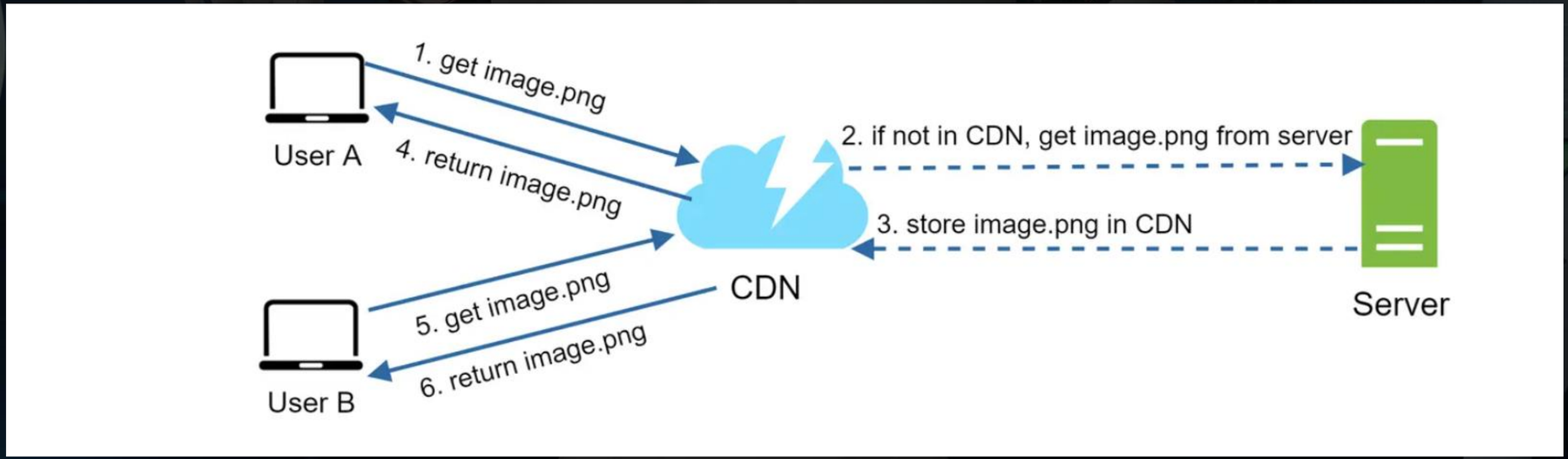
After receiving a request, a web server first checks if the cache has the available response. If it has, it sends data back to the client. If not, it queries the database, stores the response in cache, and sends it back to the client. This caching strategy is called a read-through cache

```
SECONDS = 1
cache.set('myKey, 'hi there', 3600 * SECONDS)
cache.get('myKey')
```

- Use cache for frequently read, rarely updated data — ideal for boosting speed without risking stale writes.
- Avoid caching critical or persistent data, as cache is stored in volatile memory and is lost on server restart.
- Set an expiration policy to auto-remove outdated cache entries — balance between too short (frequent DB reloads) and too long (stale data).
- Maintain consistency between cache and database — especially tricky when writes occur or in multi-region systems.
- Prevent single points of failure (SPOF) by using multiple cache servers across data centers.
- Overprovision memory in cache nodes to handle unexpected spikes and avoid crashes due to full memory.

7. CONTENT DELIVERY NETWORK


Client
Client
120ms
Origin
30ms
CDN


1. get image.png
User A
4. return image.png
2. if not in CDN, get image.png from server
3. store image.png in CDN
CDN
Server
5. get image.png
6. return image.png
User B

CDN

- A CDN (Content Delivery Network) is a group of globally distributed servers used to deliver static content like images, CSS, JS, and videos.
- CDN servers cache static files close to end users, reducing the time it takes to load websites.
- When a user visits a website, the nearest CDN server delivers static content, improving speed and reducing latency.
- Geographic distance matters: The closer the user is to a CDN node, the faster the load time.
- CDNs significantly boost performance and scalability, especially for websites with global traffic.

How it Works

1. User A tries to get image.png by using an image URL. The URL's domain is provided by the CDN provider. The following two image URLs are samples used to demonstrate what image URLs look like on Amazon and Akamai CDNs:

- https://mysite.cloudfront.net/logo.jpg
- https://mysite.akamai.com/image-manager/img/logo.jpg

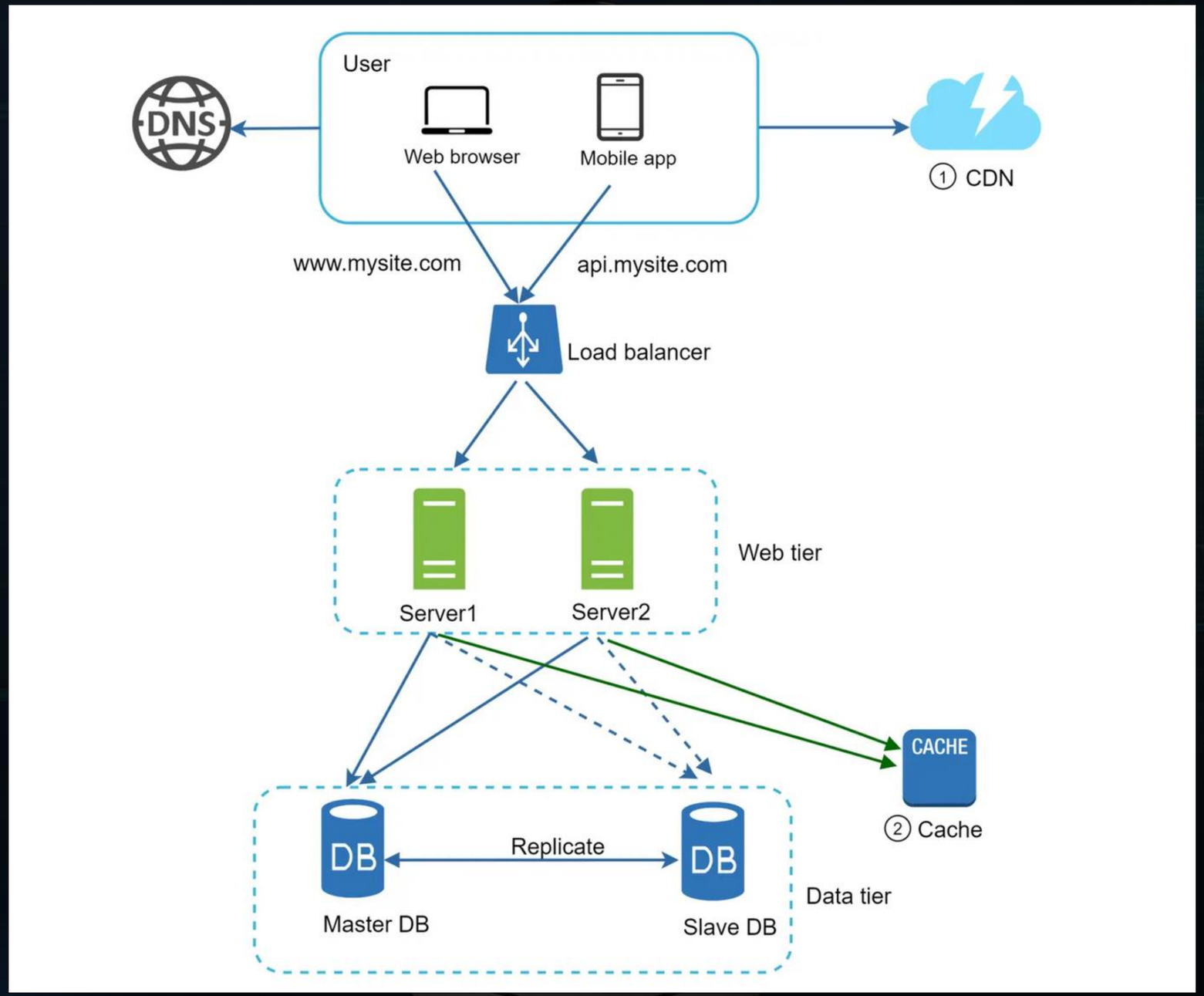
2. If the CDN server does not have image.png in the cache, the CDN server requests the file from the origin, which can be a web server or online storage like Amazon S3.

3. The origin returns image.png to the CDN server, which includes optional HTTP header Time-to-Live (TTL) which describes how long the image is cached.

4. The CDN caches the image and returns it to User A. The image remains cached in the CDN until the TTL expires.

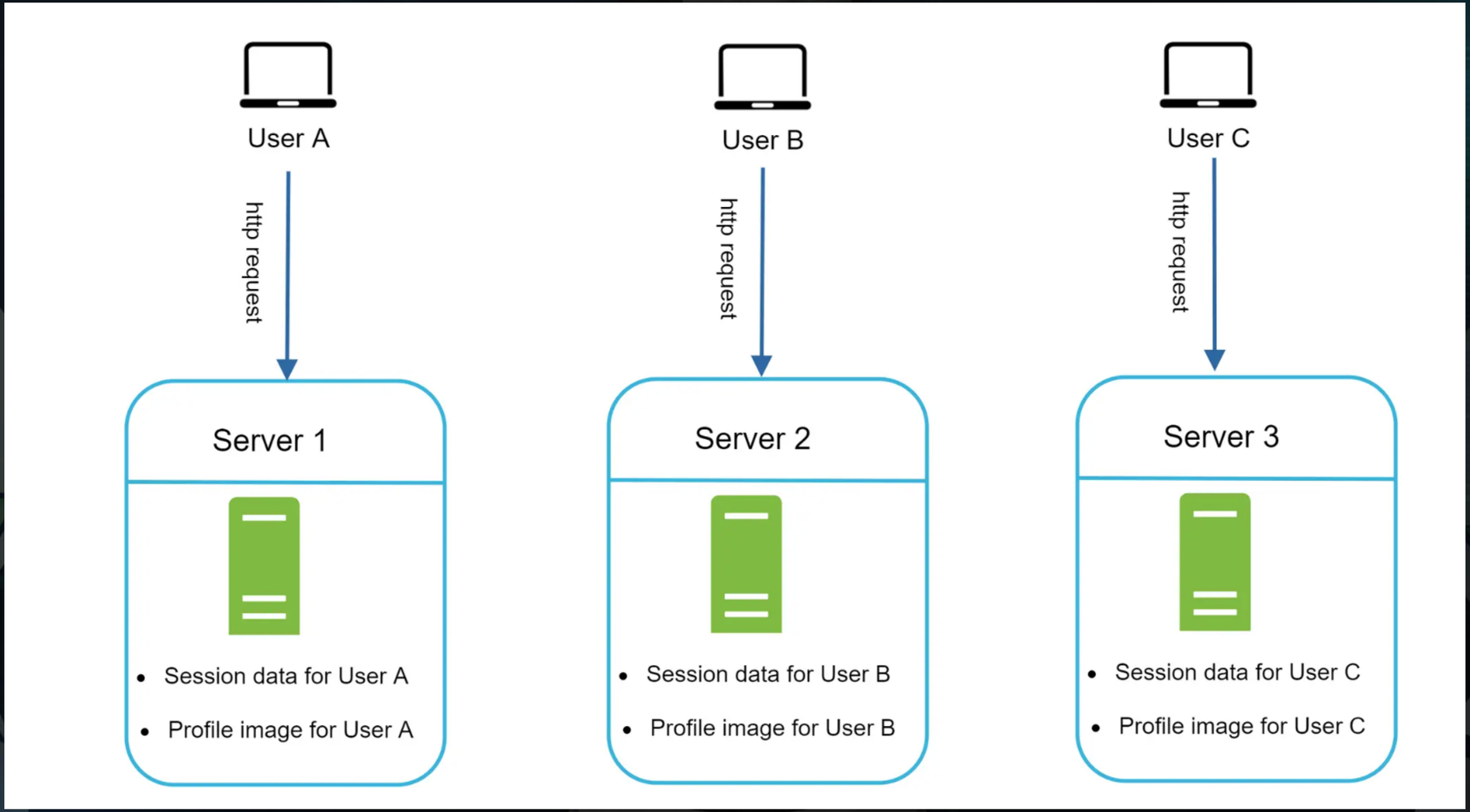
5. User B sends a request to get the same image.

6. The image is returned from the cache as long as the TTL has not expired.

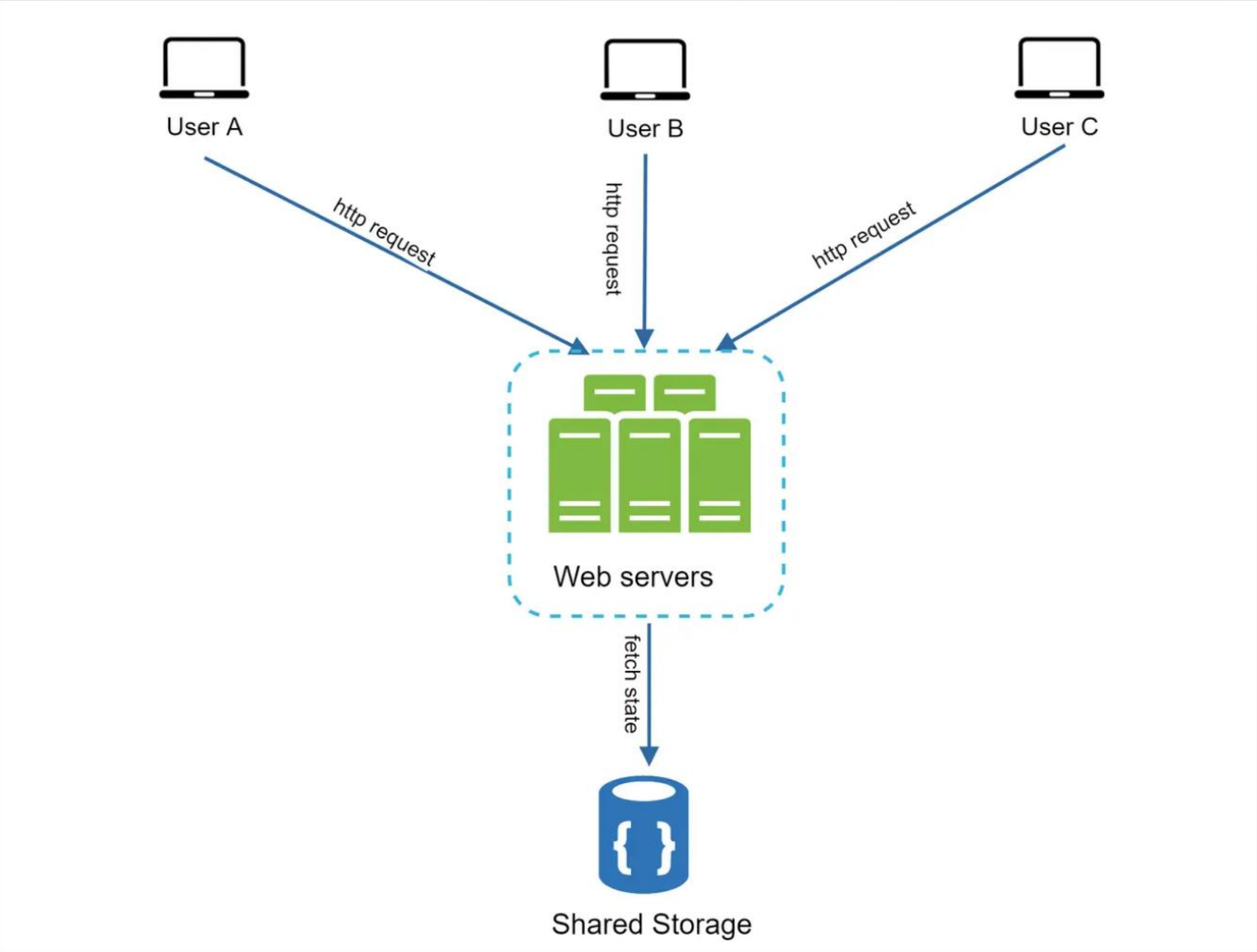

User
DNS
Web browser
Mobile app
① CDN
www.mysite.com
api.mysite.com
Load balancer
Web tier
Server1
Server2
CACHE
② Cache
DB
Replicate
DB
Master DB
Slave DB
Data tier

8. STATELESS WEB TIER

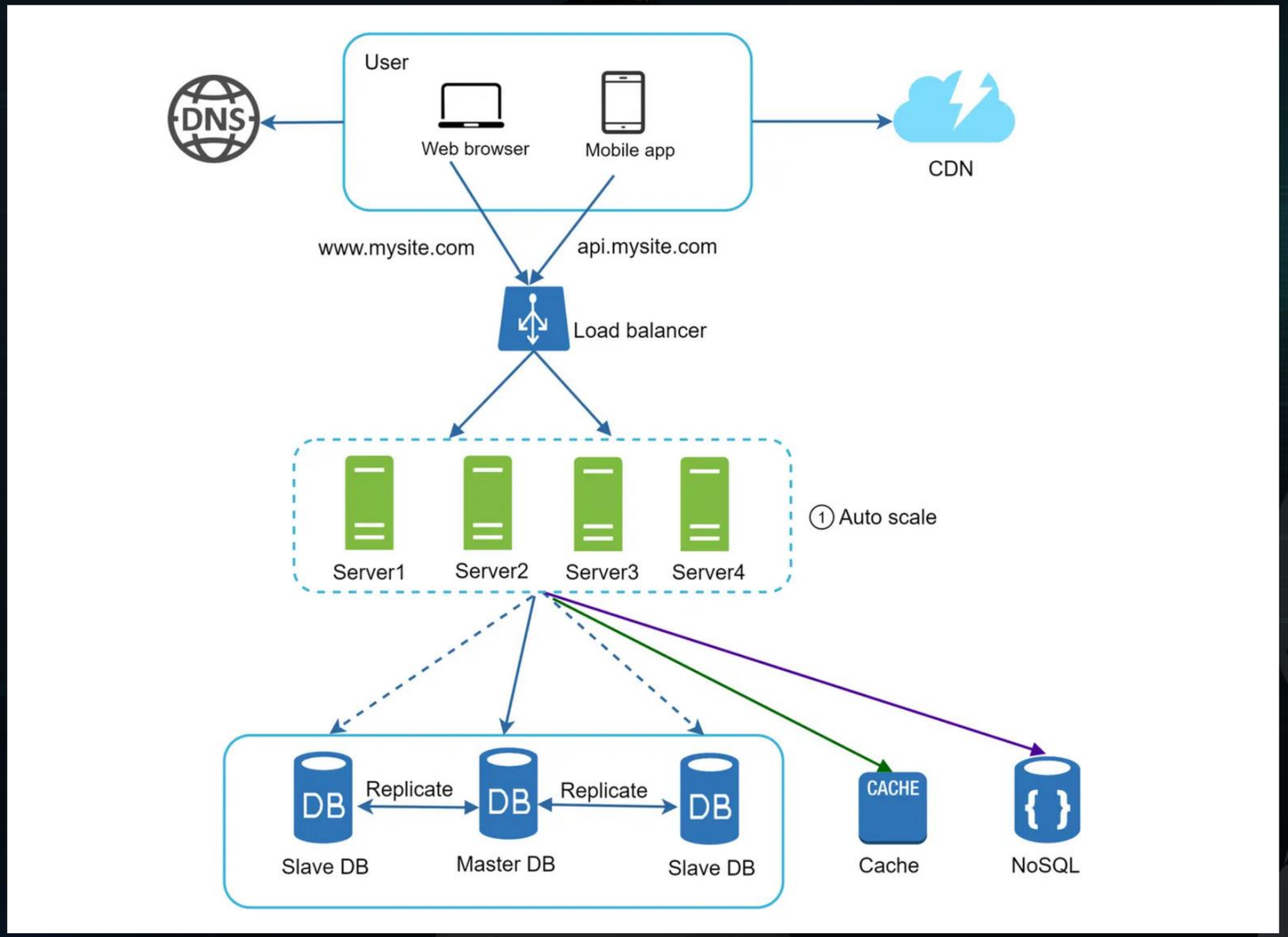
STATEFUL ARCHITECTURE



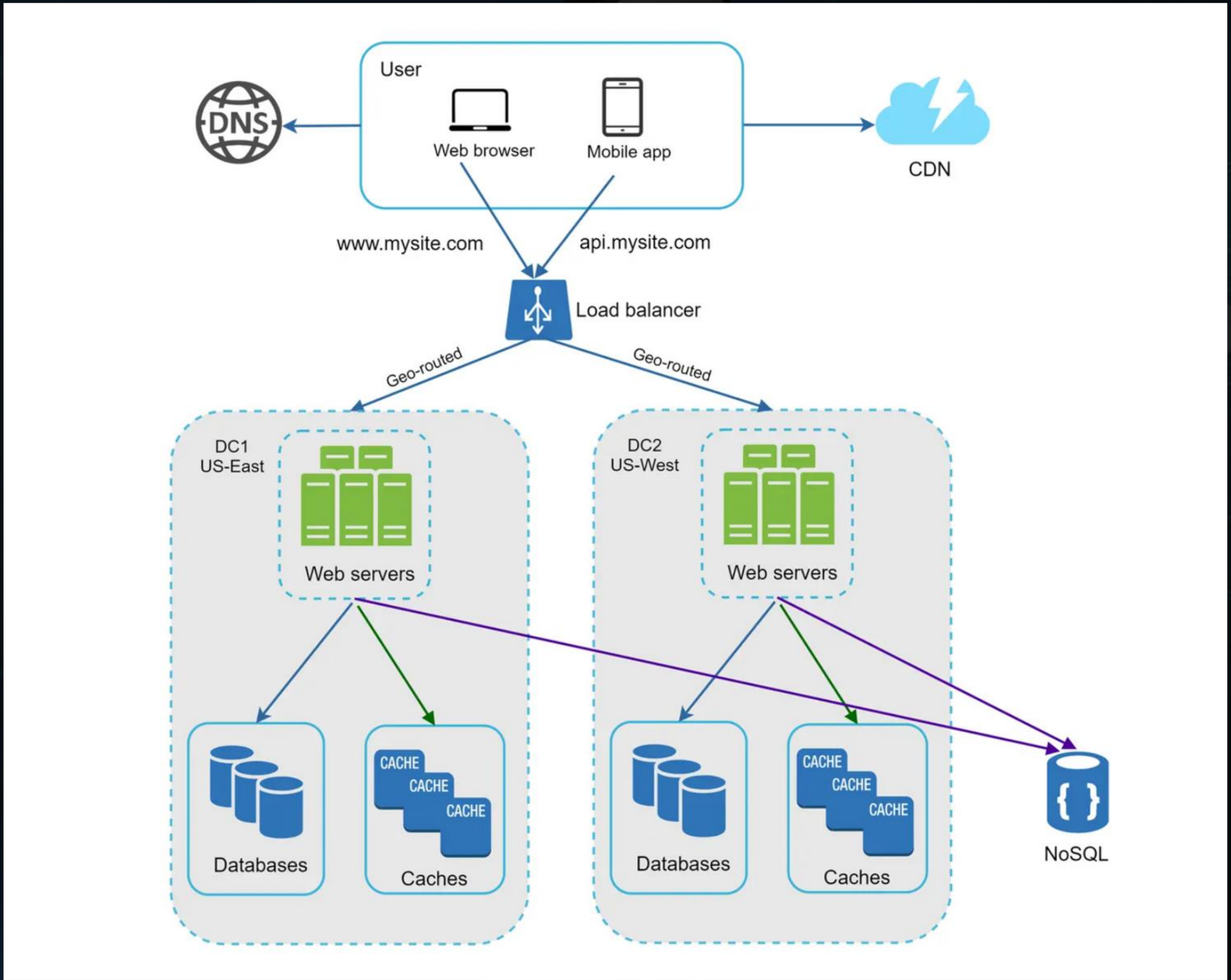
STATELESS ARCHITECTURE

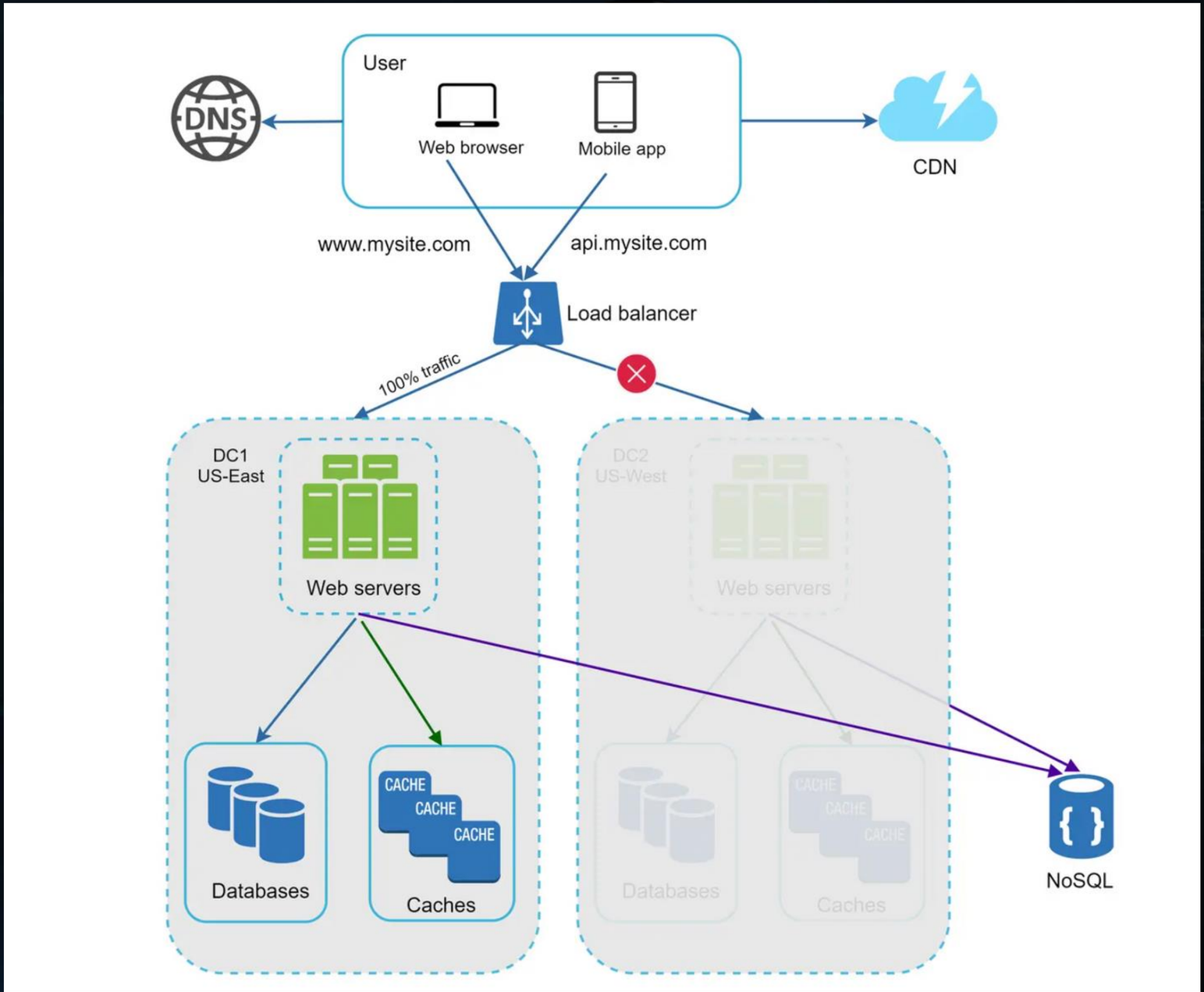


- To scale web servers horizontally, session/state data (like user login info) must be moved out of individual servers.
- Stateful servers store session data locally — requiring all of a user's requests to hit the same server (using sticky sessions).
- Stateful setups complicate scaling, as adding/removing servers or handling failures becomes difficult.
- Stateless servers do not store user session data locally — instead, all state is kept in a shared data store (DB or NoSQL).
- In a stateless system, any user request can go to any server, improving flexibility, fault tolerance, and load distribution.
- Stateless architecture is simpler, more scalable, and more resilient, making it the preferred choice for large-scale systems.


User
DNS
Web browser
Mobile app
CDN
www.mysite.com
api.mysite.com
Load balancer
Server1
Server2
Server3
Server4
1 Auto scale
DB
Replicate
DB
Replicate
DB
Slave DB
Master DB
Slave DB
CACHE
Cache
NoSQL

9. DATA CENTERS


User
Web browser
Mobile app
DNS
CDN
www.mysite.com
api.mysite.com
Load balancer
Geo-routed
Geo-routed
DC1
US-East
Web servers
DC2
US-West
Web servers
Databases
CACHE
CACHE
CACHE
Caches
Databases
CACHE
CACHE
CACHE
Caches
NoSQL


User
DNS
Web browser
Mobile app
CDN
www.mysite.com
api.mysite.com
Load balancer
100% traffic
DC1
US-East
Web servers
Databases
CACHE
CACHE
CACHE
Caches
DC2
US-West
Web servers
Databases
CACHE
CACHE
CACHE
Caches
NoSQL

10. MESSAGE QUEUE

A Message Queue is a system that allows asynchronous communication between different parts of a software system by sending and receiving messages through a queue.

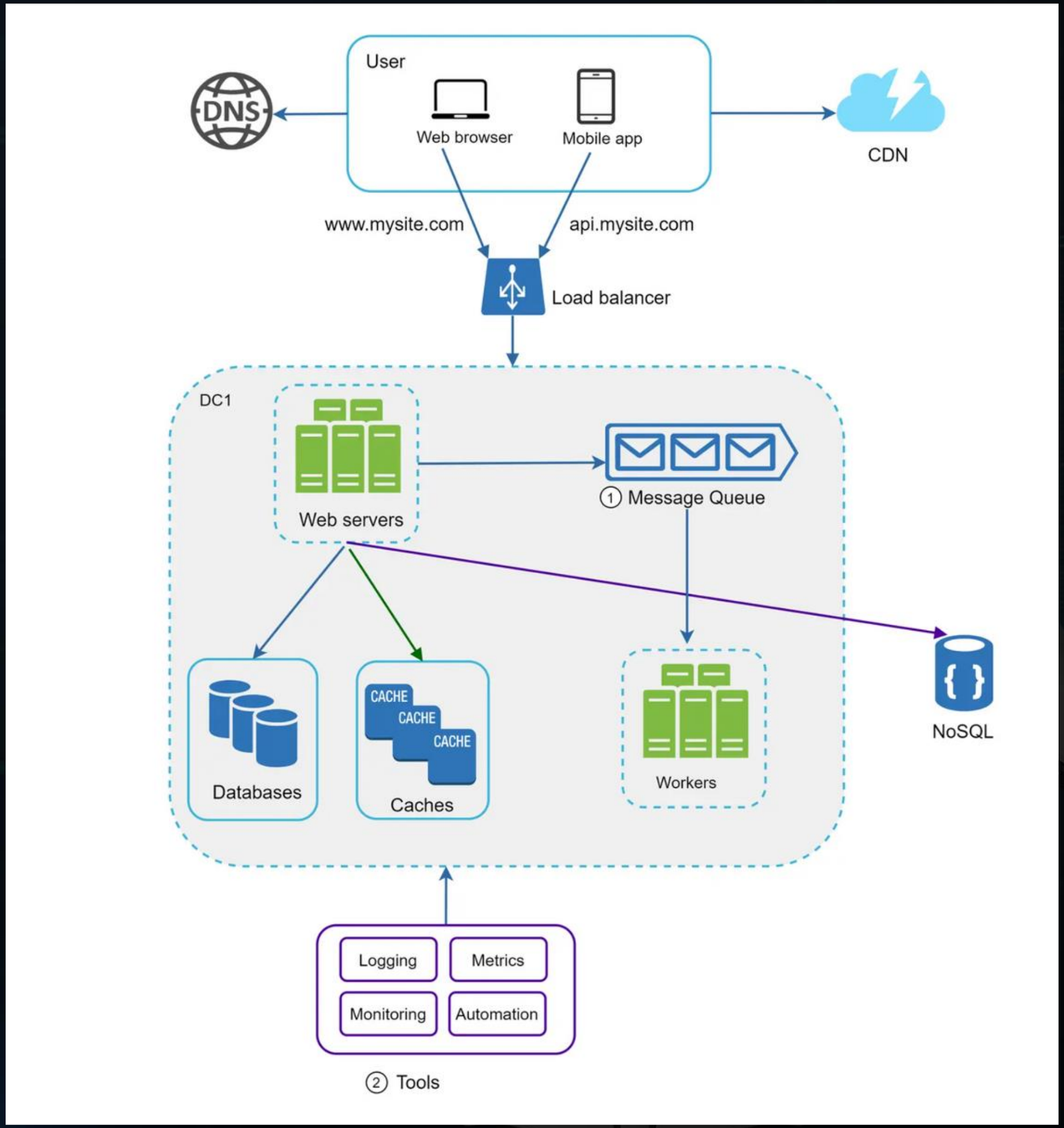
It helps decouple components — meaning systems don't need to run at the same time or speed.

For example: a website can quickly accept a request and queue it, and a worker can process it later.

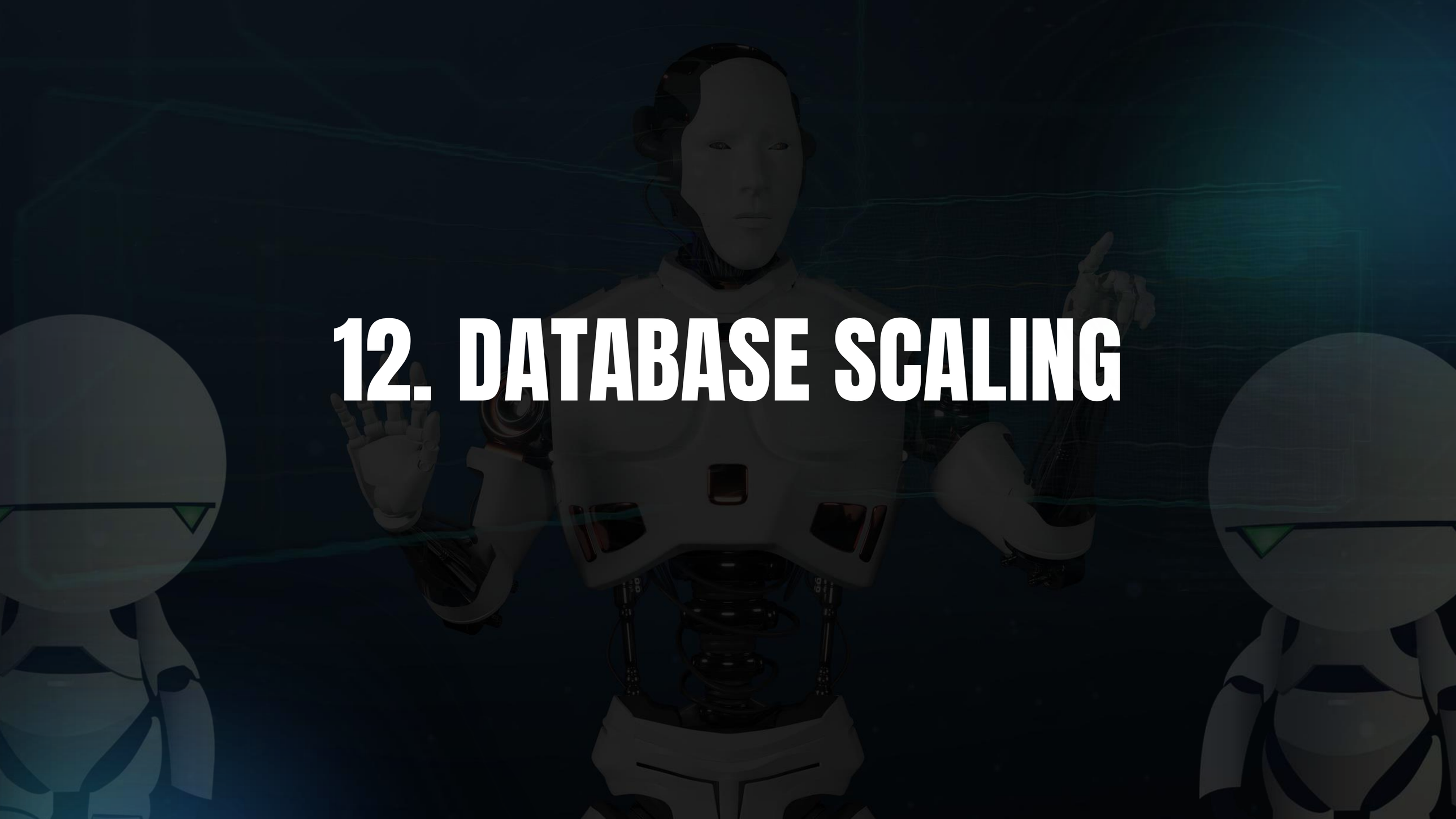
How It Works (Step-by-Step):

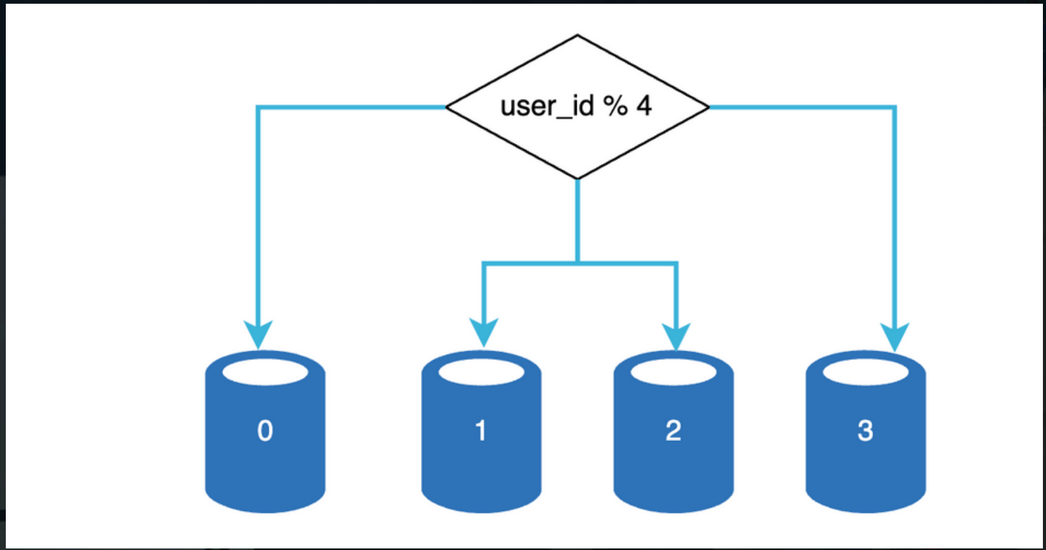
1. Producer (or Publisher) creates a task (message)
2. The message is sent to a Queue (like a buffer or waiting line)
3. One or more Consumers (or Subscribers) pull tasks from the queue and process them
4. Once processed, the task is marked as done or removed

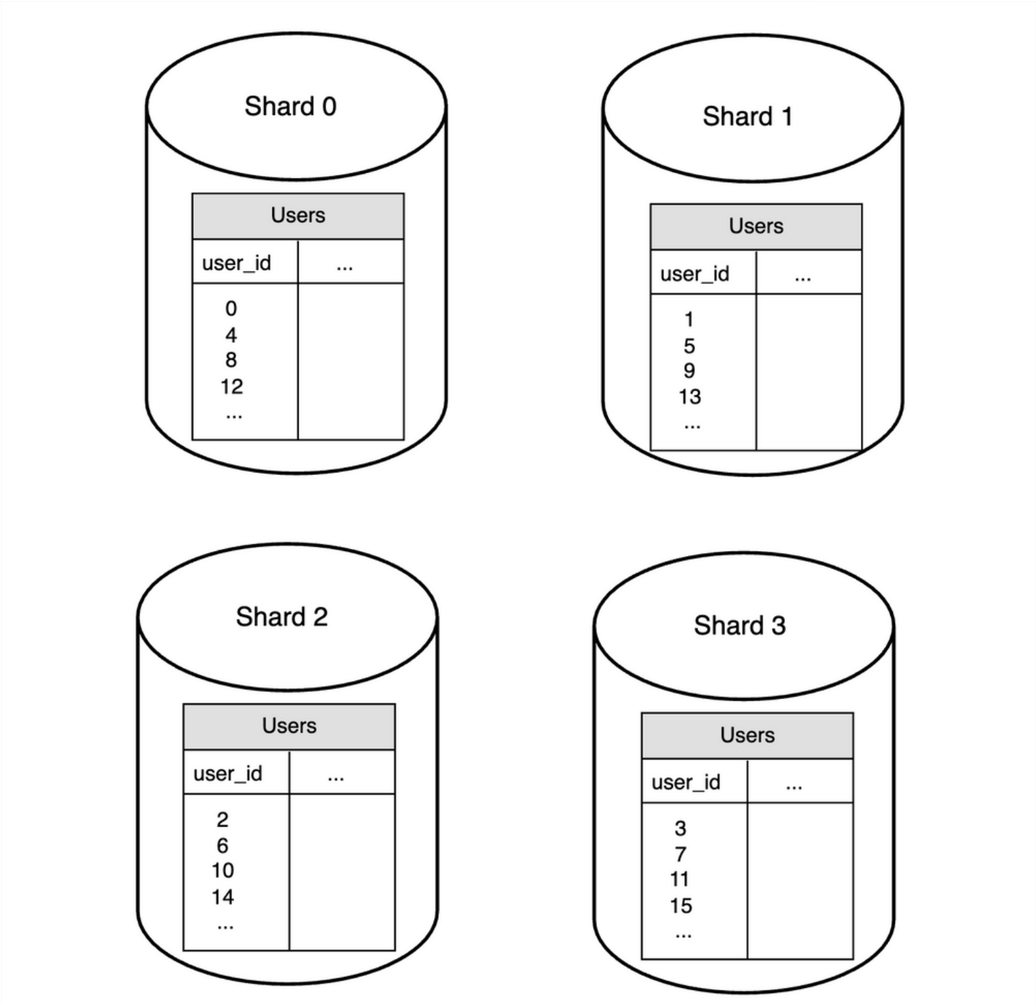
11. LOGGING, METRICS AND AUTOMATION


User
Web browser
Mobile app
DNS
CDN
www.mysite.com
api.mysite.com
Load balancer
DC1
Web servers
1 Message Queue
Databases
CACHE
CACHE
CACHE
Caches
Workers
NoSQL
Logging
Metrics
Monitoring
Automation
2 Tools

12. DATABASE SCALING




user_id % 4
0
1
2
3

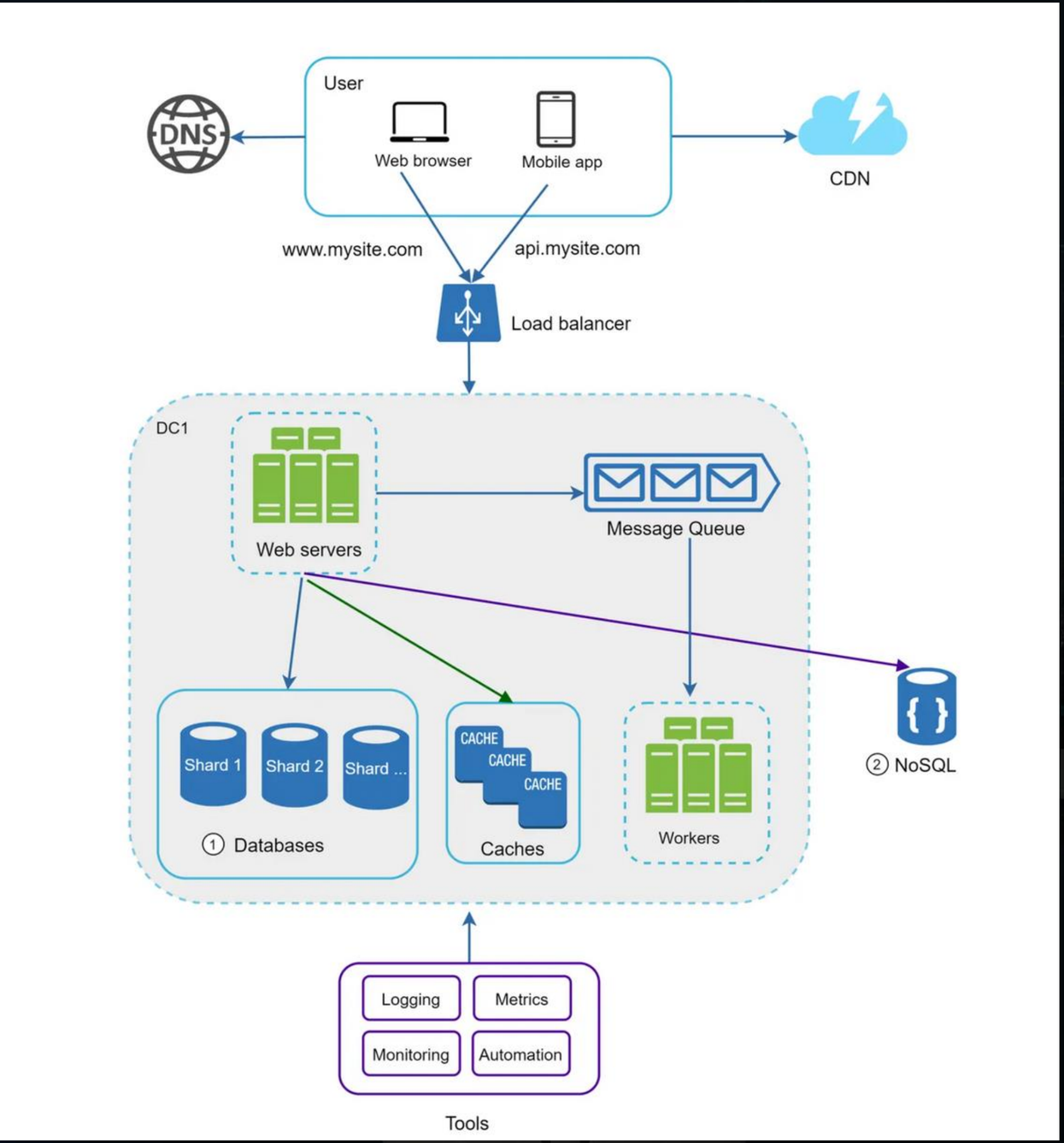

Shard 0
Users
user_id
...
0
4
8
12
...
Shard 1
Users
user_id
...
1
5
9
13
...
Shard 2
Users
user_id
...
2
6
10
14
...
Shard 3
Users
user_id
...
3
7
11
15
...

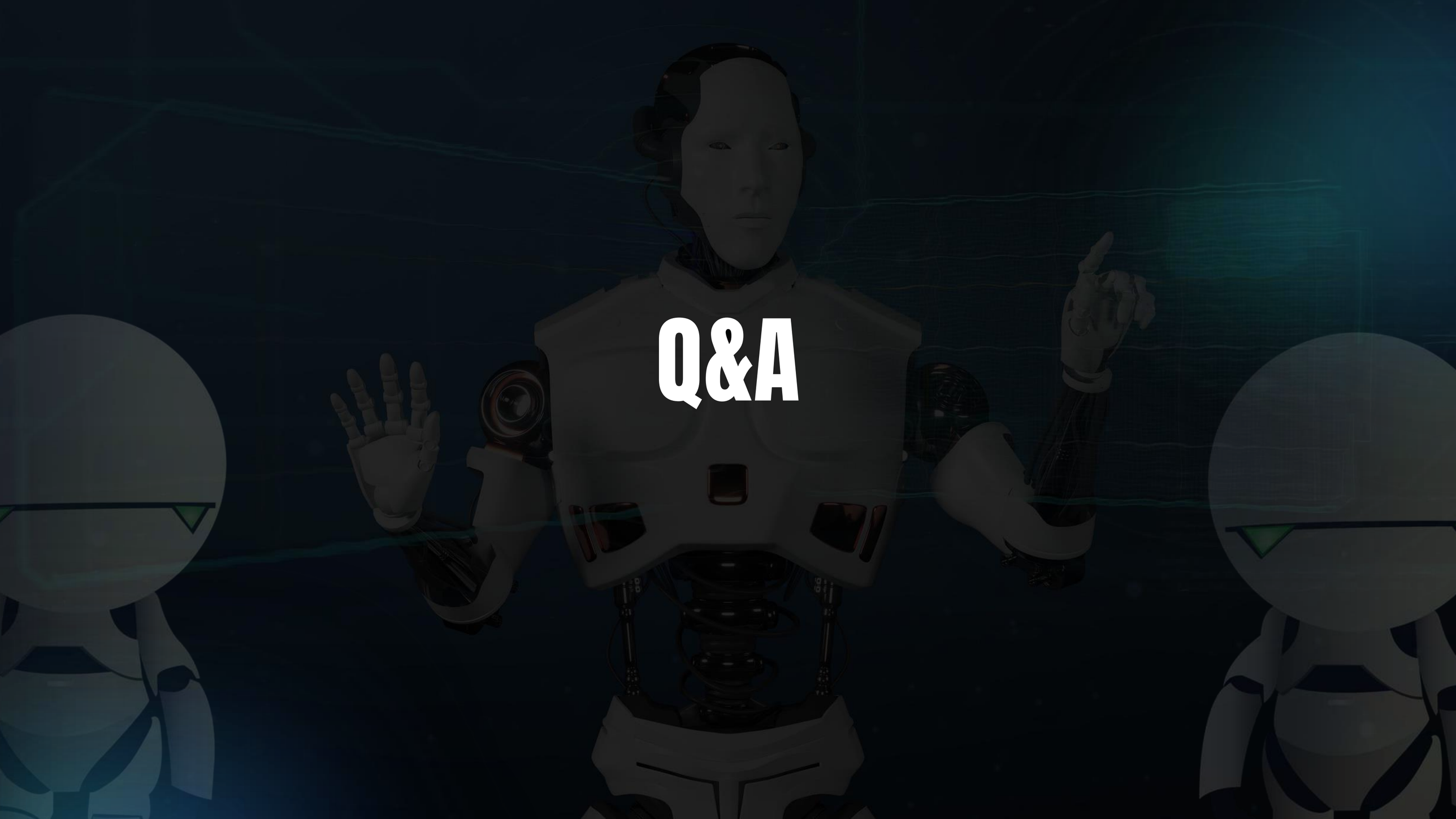
Sharding/Partition

Sharding is a technique to split a large database into smaller, faster, more manageable parts, called shards.
Each shard has the same database structure (schema) but contains a different subset of the data.
It helps systems scale horizontally and handle large volumes of traffic or data.

1. Sharding splits large databases into smaller pieces, called shards, for easier management and better performance.
2. Each shard has the same schema, but stores unique data, allowing parallel processing.
3. A hash function (e.g., user_id % 4) is used to decide which shard stores or retrieves a given piece of data.
4. Sharding reduces database load, improves query performance, and helps scale systems horizontally.
5. It prevents bottlenecks by spreading data and requests across multiple servers.
6. Commonly used in large-scale applications (e.g., social media, e-commerce) where data volume grows rapidly.
7. Adding or removing shards can be complex — requires careful shard key design to avoid data imbalance.

FINAL DESIGN


User
Web browser
Mobile app
DNS
CDN
www.mysite.com
api.mysite.com
Load balancer
DC1
Web servers
Message Queue
Shard 1
Shard 2
Shard ...
1 Databases
CACHE
CACHE
CACHE
Caches
Workers
2 NoSQL
Logging
Metrics
Monitoring
Automation
Tools


Q&A